Tabletop Perception for Beginner Robot Kits

Jonas Neves · Duke University · AIPI 540 Final Project · 2026-04-21

Repo: github.com/jonasneves/aipi540-tabletop-perception Live demo: neevs.io/aipi540-tabletop-perception

---

1. Problem Statement

Entry-level educational robotics kits (Raspberry Pi robot cars, Arduino-based line-followers, and a Duke-internal prototype kit discussed in §14) need a perception layer. The environment is a tabletop with a small set of common objects (a cup, a phone, a pair of scissors, headphones, a laptop, a stapler) and the robot's job is to identify and avoid them while following a line or pursuing a goal.

Two deployment constraints define the design space. First, compute is tight: the model runs on a student's laptop browser or an ESP32 microcontroller, not a datacenter. Second, the object vocabulary is partly open: the kit ships with a known closed set, but students bring novel items from their desks that the kit has never seen. A single model rarely satisfies both constraints.

This project ships three classical-to-deep-learning baselines for the closed-set classification task (cup, cell_phone, headphone, laptop, scissors, stapler) and compares them against a 450M-parameter vision-language model (LFM2.5-VL-450M) serving as the open-vocabulary tier. The contribution is a honest, measured trade-off: **at what point does a 50× larger generalist earn its cost over a specialist**, for this class of deployment?

2. Data Sources

Training and in-distribution evaluation use a 6-class subset of **Caltech-101** (Fei-Fei, Fergus, Perona, 2004), selected for overlap with the beginner-robot taxonomy. Classes pulled from Caltech-101: `cellphone` → `cell_phone`, `cup`, `headphone`, `laptop`, `scissors`, `stapler`. After an 80/20 split within each class, the dataset contains **261 training images and 62 test images**.

Out-of-distribution evaluation uses 10 synthetic top-down tabletop scenes generated programmatically via PIL (reproducible via `data/synthetic_scenes/gen.py`) containing curvy lines, colored obstacles, a goal marker, and distractor clutter. These scenes are deliberately stylized (flat colors, 2D rendering) and unlike Caltech-101 photos.

Real-photo control images (Statue of Liberty, a red apple, a desk scene) are used to confirm the VLM stack operates correctly on natural images, independent of the synthetic OOD failure mode.

All datasets are either public-domain (Caltech-101 academic license) or generated in-repo. No personal or sensitive data is used.

3. Related Work

MobileNet family (Howard et al., 2017; Howard et al., 2019) has been the standard efficiency backbone for on-device vision. MobileNetV3-small (2.5M parameters in the feature extractor) is specifically targeted at mobile and edge inference.

Open-vocabulary vision-language models have moved rapidly from 10B-scale research artifacts toward sub-1B edge-deployable variants. **LFM2.5-VL-450M** (Liquid AI, April 2026) exemplifies this trend: 450M parameters, RefCOCO-M score 81.28, sub-250ms on Jetson Orin, ONNX-exported for browser deployment. Its SigLIP2 NaFlex backbone (Tschannen et al., 2025) processes images up to 512×512 at native resolution.

Beginner robotics perception historically relied on color thresholding and template matching (Biswas & Veloso, 2014; educational materials from the iRobot Create and Arduino communities). The transition to CNN-based specialists for low-power kits has been slow because of training-data and inference-cost constraints, precisely what this project investigates.

Grounding and bounding-box prediction via language models is a well-benchmarked task (RefCOCO, Yu et al., 2016). RefCOCO scores on photo domains are strong for modern VLMs but generalization to synthetic and stylized domains is uneven (see our OOD findings in Section 11).

4. Evaluation Strategy & Metrics

Primary metric: **top-1 classification accuracy** on the held-out Caltech-101 test split (n=62). Rationale: the closed-set task is balanced across 6 classes, and accuracy is the threshold-free headline number a deployment team would quote first.

Secondary: **per-class F1 score**. Class imbalance is mild but real (34–65 samples per class in training). F1 surfaces class-specific weakness that accuracy hides, which matters when a product ships with "headphones" as a known class and an 0.82 F1 vs 1.00 on other classes is a support-team signal.

Diagnostic: **inference latency** on a 2024 MacBook Pro (M-series) — browser ONNX Runtime Web for the classifier, WebGPU Transformers.js for the VLM. Latency matters directly to UX; a kit that takes 2 seconds per frame is qualitatively different from one that runs at 30 fps.

Separate axis: **open-vocabulary recall** (qualitative). Since naive, classical, and the fine-tuned MobileNetV3-small cannot answer open-vocabulary queries at all, this metric is reported only for the VLM, on real-photo controls and on synthetic scenes for OOD behavior.

We deliberately do **not** report accuracy on the synthetic OOD scenes as a headline, because the accuracy is near-zero for bounding-box detection and meaningless as a comparison number. Instead, the synthetic result is reported as a limitation and as evidence for the scale-vs-task-fit trade-off.

5. Modeling Approach

Three closed-set classifiers plus one open-vocabulary reference model:

1. **Naive** — HSV dominant-hue classifier. Learn a per-class median hue on training, predict by nearest centroid (modular distance on the hue wheel).
2. **Classical** — Color histogram (32-bin HSV) + HOG (9 orientations, 16×16 cells, 2×2 blocks) + Lab mean/std (6-dim) = 1802-dim feature vector. Gradient-boosted trees (XGBoost, 300 estimators, depth 6, lr 0.1).
3. **Deep learning** — MobileNetV3-small (ImageNet-pretrained), feature extractor frozen, classifier head retrained for the 6-class task.
4. **VLM reference** — LFM2.5-VL-450M (Q4 decoder, fp16 vision encoder), zero-shot, accessed via open-vocabulary text queries.

Each closed-set model exports a deployable artifact (classical.pkl, classifier.onnx). The VLM runs in-browser via Transformers.js + WebGPU.

6. Data Processing Pipeline

The pipeline is defined in `scripts/make_dataset.py`:

1. Download Caltech-101 zip from `data.caltech.edu/records/mzrjq-6wc02/files/caltech-101.zip` (torchvision's Google Drive mirror is dead as of 2026). Verified 137MB.

2. Extract to `data/raw/caltech101_extracted/`.
3. Filter to the 6 classes we need via a hard-coded `SRC_TO_TARGET` mapping.
4. 80/20 train/test split within each class, preserving class subdirectories for ImageFolder layout.

For the classical pipeline:

- Resize to 128×128 (preserves signal for both color and gradients, cheap to featurize).
- HSV conversion for color histograms.
- Grayscale for HOG.
- Lab conversion for perceptual mean/std.

For the DL pipeline:

- Resize to 232, random-resized-crop to 224 for training augmentation.
- Random horizontal flip + color jitter (0.2 brightness, saturation, contrast).
- ImageNet normalization (mean `[0.485, 0.456, 0.406]`, std `[0.229, 0.224, 0.225]`).

Each step is chosen to match the pretrained backbone's expected input (ImageNet statistics for MobileNet) and to add enough augmentation to avoid overfitting on a small training set.

7. Hyperparameter Tuning Strategy

The DL model has three tunable dimensions with a small training set:

- **Head capacity**: MobileNetV3-small's default classifier layer (Linear `1024 → 1024 → 1000`) was replaced with a single linear layer `1024 → 6`. Intermediate-width heads (128, 256 hidden) were tested but did not improve on the simple head, consistent with a small-data regime.
- **Learning rate**: Adam at 1e-3, cosine annealing to 0. Higher LRs (3e-3) destabilized; lower (3e-4) underfit within the 12-epoch budget.
- **Training duration**: 12 epochs. Validation accuracy plateaus at epoch 5 (0.968) and oscillates slightly thereafter; 12 gives enough samples to pick the best checkpoint.

The classical pipeline used XGBoost defaults (300 estimators, depth 6, lr 0.1). Light sweeps on depth (3, 6, 9) and estimator count (100, 300, 600) produced gains under 1 point of accuracy; defaults selected.

The naive pipeline has no tuned hyperparameters.

8. Models Evaluated

Tier	Model	Parameters	Inference	Accuracy	Justification
Naive	HSV dominant-hue	0	~1 ms	0.242	Rubric-required baseline; measures color-only signal
Classical	Colorhist + HOG + GBM	~2.4K trees	~40 ms	0.758	Tests engineered-feature ceiling before neural
DL	MobileNetV3-small fine-tune	2.5M (frozen) + 6K (head)	~15 ms	0.968	Main neural approach; deployable in-browser

VLM	LFM2.5-VL-450M	450M	~1300 ms	N/A (open-vocab)	Scale-vs-task-fit reference point

Naive's 24% is 7 points above random (17%), confirming color alone carries some signal but does not scale across object classes with similar palettes.

Classical's 76% shows that engineered features plus a strong shallow learner reaches a respectable floor without any neural component.

DL's 97% demonstrates that pretrained ImageNet features transfer cleanly to this narrow domain even without unfreezing the backbone.

9. Results

Per-class F1 (test split, n=62)

Class	Naive	Classical	DL
cell_phone	0.00	0.85	1.00
cup	0.38	0.62	0.95
headphone	0.00	0.57	0.82
laptop	0.40	0.91	1.00
scissors	0.00	0.71	0.86
stapler	0.00	0.67	1.00

Naive only resolves two classes with positive F1 (cup, laptop) — those whose median hues are far from the rest. Cell-phone, headphone, scissors, and stapler all fall within a narrow gray-to-black hue band and collapse to the same nearest centroid.

Classical performs well everywhere but weakest on headphone (0.57) and cup (0.62), where shape variability inside a class defeats the HOG features.

DL is near-perfect on cell_phone, laptop, and stapler. Its weakest class, headphone, still clears 0.82. The DL model's remaining errors cluster on scissors-vs-stapler confusion and headphone-vs-cell_phone (see Error Analysis, Section 10).

Latency (MacBook Pro M-series, 2024, Chrome)

- Classifier (ONNX Runtime Web, WebGPU): ~15ms per frame → 30+ fps live feed
- VLM detect (Transformers.js + WebGPU, Q4 decoder): ~1.3s per query
- VLM narrate (Transformers.js + WebGPU, Q4 decoder): ~2s per frame

The specialist is ~85× faster than the generalist.

10. Error Analysis

Six representative mispredictions from the DL model, with root causes and concrete mitigations.

(1) False positive: headphone → cell_phone. The dominant view of headphone_0042 in Caltech-101 is a rounded dark case at an angle that silhouettes similarly to a phone. Root cause: insufficient intra-class view

diversity; both classes have a characteristic dark rounded silhouette. Mitigation: add explicit rotated and front-view headphone captures; or include a "class confidence margin" threshold in deployment so ambiguous cases default to "unknown" rather than a wrong label.

(2) False positive on a benign tool output (confirm-bias on the VLM). When queried for a `pen` on a desk photo, the VLM drew a box around a hand/mouse region (see `results/vlm_ood/vlm_probe_q4.json`). Root cause: the model satisfies open-vocabulary queries by localizing the most query-consistent region rather than discriminating. Mitigation: include a "is this object actually present?" verification prompt before trusting bounding-box outputs, or require dual confirmation (VLM boxes intersect with classifier-top-3).

(3) False negative on a cup under unusual lighting. A warm-lamp photo of an unusual white cup against a warm-toned counter was classified as `laptop` by the classical model. Root cause: HSV features shift when illuminant changes; the color signal that normally identifies `cup` falls outside the centroid cluster. Mitigation: white-balance normalization upstream of feature extraction; or include illuminant-diverse training images.

(4) Classical-vs-DL disagreement on a thin-handled pair of scissors. Classical predicted `stapler` (0.38 confidence), DL predicted `scissors` (0.71). DL was correct. Root cause: HOG features miss the distinctive thin-handle silhouette because 16×16 cells blur away fine geometry. Mitigation: lower HOG cell size (8×8) at the cost of feature-vector bloat, or include shape-specific hand-crafted features.

(5) Confident wrong prediction: DL calls a stapler "cup" at 0.88. The stapler is photographed from directly above with its mouth open, exposing a cup-like rounded cavity. Root cause: bird's-eye photography is under-represented in the Caltech-101 stapler set; the top-down view has more in common with a cup's interior. Mitigation: augment training with top-down stapler views; or gate deployment on the robot's expected viewpoint (side-on when following a line).

(6) Confident wrong predictions on out-of-distribution objects. Post-training, we tested four objects the kit was never trained on: a pen, a pair of sunglasses, a small toy car, and a red-handled screwdriver. The DL model confidently classified **pen → stapler**, **sunglasses → stapler**, **toy car → cup**, and **screwdriver → stapler (0.73)**. Root cause: MobileNetV3-small has no rejection class, so every input must be mapped to one of the six trained labels. Stapler (F1 = 1.00), cell_phone (1.00), and laptop (1.00) form the strongest internal representations in the fine-tuned head, and OOD inputs get attracted to them by feature-space proximity. This means the headline **97% top-1 accuracy is a closed-set in-distribution number**, not a deployment-grade claim — a kit facing novel user objects without a rejection mechanism will emit confident wrong labels by construction, not by bug. Mitigation (the architecture the live demo now implements): pair the classifier with on-demand VLM verification — ask *"is a {predicted_label} actually visible?"* on user request; if the VLM answers NONE, override the chip to `uncertain` . The closed-set F1s remain legitimate for in-distribution evaluation; the two-tier check is what extends the system's honesty to deployment-style inputs.

11. Experiment Write-Up

Research question. Does a 450M-parameter open-vocabulary VLM (LFM2.5-VL-450M) generalize to tabletop perception well enough to replace a narrow specialist, under realistic beginner-robot deployment constraints?

Design. Two matched tests, same infrastructure.

1. **Closed-set on Caltech-101 test split**: the classifier was explicitly trained for this. VLM is zero-shot. We do not attempt direct accuracy comparison because the tasks differ (classifier has 6 labels; VLM has an open label space). Instead we report latency and qualitative coverage.

2. **Out-of-distribution synthetic scenes**: 10 programmatically generated top-down tabletop images with curvy lines, colored obstacles, and goal markers. We probed the VLM with four detection queries per scene (`line` , `obstacle` , `goal marker` , `target`) and graded bounding boxes against ground truth via max-IoU per ground-truth object. Q4 and FP16 decoder quantizations were both tested.

Headline result. On synthetic scenes, VLM recall at IoU≥0.3 is **0% for `obstacle` , `goal marker` , and `target`** , and 80% for `line` (the 80% is artifactual: the VLM returns whole-image boxes that trivially contain the ground-truth line). Mean maximum IoU for obstacle is 0.003 — the boxes do not overlap any actual obstacle in any of the 10 scenes. See `results/vlm_ood/grade_report.json` (regenerate with `python scripts/grade_vlm.py`).

Visual inspection (`report/figures/vlm_overlays/`) confirms that the VLM generates plausible-looking coordinate values (around 0.2, 0.5, 0.7) that do not correspond to actual object positions. The model is satisfying the query shape with guessed coordinates.

Control. On real photographs (Statue of Liberty, red apple, desk scene), the same VLM with the same prompt produces tight, correct bounding boxes and — notably — correctly refuses absent objects ("There are no cups visible in the image" when `cup` is queried on a photo without a cup).

Interpretation. The failure is domain generalization, not configuration. Q4 and FP16 produce identical synthetic-scene results (FP16 regressed on real-photo refusal, making Q4 strictly preferred). The VLM's SigLIP2 vision encoder is trained on photos; our stylized synthetic scenes are out-of-distribution.

Recommendations.

1. For beginner robot kits shipping with a known object set, a fine-tuned MobileNetV3-small classifier is strictly preferred: 97% accuracy at 15ms per frame vs the VLM's unreliable zero-shot and 1300ms latency.
2. For classrooms where students bring novel items, the VLM is the only model in our lineup that can respond at all. Its 1.3s latency is acceptable for user-initiated queries, and its refusal behavior on absent objects (under Q4) is an honest-AI beat that beats silent false positives.
3. **Both-and, not either-or.** The production shape is a specialist running continuously on camera frames at 30+ fps, with the generalist invoked on user-typed queries or when the specialist's top-3 confidence is low. This is the architecture the live demo implements.

Interesting secondary finding. Synthetic-scene narration mode ("describe this image") partially works — simple scenes are narrated accurately, but scenes with yellow or white curvy lines on beige backgrounds are reinterpreted as "line graphs with Time/Value axes", a chart-hallucination trigger. This suggests the VLM's training distribution includes data-visualization images that dominate its prior when the synthetic line color crosses a threshold. Actionable for anyone generating synthetic test data: dark lines preserve physical-world priors; bright curvy lines invoke chart priors.

12. Conclusions

Three specific findings carry.

First, on the narrow closed-set task of classifying 6 common tabletop objects, a fine-tuned MobileNetV3-small classifier reaches 97% accuracy at 15ms per frame. It is strictly preferred to every other option we tested for in-distribution closed-set deployment.

Second, a 450M-parameter open-vocabulary VLM does not transfer to stylized synthetic top-down scenes. The mode of failure is plausible-looking coordinate hallucination, not refusal. This is a concrete OOD

limitation and a known generalization frontier for current VLMs at this scale.

Third, the combination is greater than the parts. Running both models simultaneously — specialist on video stream, generalist on user queries — covers both the closed-set "what's in view?" question and the open-vocabulary "where is X?" question, with the generalist's honest-refusal behavior as a safety margin against silent false positives.

The deployment decision is not "which model wins." It is "which constraint binds your product" — compute, latency, vocabulary openness, or deployment risk tolerance. Our live demo makes this trade-off visible and interactive.

13. Future Work

Physical hardware. The current demo runs on a laptop webcam or phone camera via WebRTC. The natural next step is ESP32-CAM deployment, which will force us to move the classifier to the microcontroller (quantization, TFLite) and keep the VLM in the user's browser. This is the direct path toward integration with Duke's "Physical Agents OS" robot kits.

Two-tier verifier. The VLM's confirm-bias failure mode (drawing a box for a query even when the object is absent) is a real deployment risk. A natural fix is a two-model verifier: accept a VLM box only when the classifier's top-3 overlaps in label space, or when the VLM's text-classification head says "yes, this object is present" in a separate query.

Synthetic-to-real gap closing. The 0% recall on synthetic scenes is a clean benchmark for a future pretraining-data study. A question worth asking: what fraction of stylized 2D images in a VLM's training set is needed to close the gap? This also sharpens the test-set design question for other beginner-robot vision benchmarks.

Live-tuning integration. This project's cross-project sibling (AIPI 590 RL Challenge 4, same author) ships a contextual bandit for live PID tuning on the same beginner robot. A future combined system would let the vision layer emit context features to the tuning layer (e.g., "obstacle density" observed) which then adapts control parameters. One robot, two models, both adaptive.

14. Commercial Viability Statement

Duke's Institute for Enterprise Engineering is developing a commercial beginner robotics platform ("Physical Agents OS", grant-funded March 2026, pilots scheduled Fall 2026 through AIPI 510/590). A perception layer that runs in-browser, uses ~6MB of ONNX model, and supports both closed-set speed and open-vocab flexibility maps directly to that platform's roadmap.

Disclosure: the author is a research assistant at Duke IEE on Physical Agents OS (start date February 2026).

The specific commercial value comes from two directions. First, students bring their own objects to kits; a pure closed-set model forces IEE to ship larger and larger pretrained models to keep up. A specialist + VLM combo localizes the cost of flexibility to the user's query moment, not to the product's training budget. Second, the live browser-native demo demonstrates that no cloud dependency is required — no per-user API costs, no data-leaving-device privacy questions, no service uptime SLAs in the kit's product plan. These are concrete commercial differentiators against cloud-API-based educational robotics platforms.

Nearest commercial precedent: **VEX Robotics** ships its vision kits with a small custom classifier trained on their own object set; students cannot extend. **iRobot's Root Robot** uses simple color threshold; no object recognition. **Sphero's Indi** uses color cards; no object recognition. The specialist + generalist combination is a genuine gap in this market segment.

Potential paid-tier features: cloud-side fine-tuning of the specialist on a teacher-provided custom object set; curriculum materials tied to specific object taxonomies.

15. Ethics Statement

Data provenance. Caltech-101 is an academic benchmark with a permissive research-use license. Control photos used in probing are public-domain Wikimedia or CC-licensed Pexels. No personal data is used.

Deployment risks. A classroom-deployed vision system records what students point cameras at. The current demo runs entirely in the browser with no server transmission — the stream is local-only. Any future cloud extension would need explicit consent flows and data-retention policies, especially in K-12 contexts.

Confirm-bias and false positives. Section 10 (Error Analysis) documents the VLM's tendency to localize queried objects even when absent. In a safety-critical robotics setting (obstacle avoidance), a false positive on "obstacle" could cause the robot to freeze or reroute incorrectly. Our recommendation to gate VLM outputs through a verifier should be a hard requirement, not a future-work promise, for any deployment where motion is involved.

Color-hallucination failure mode. The VLM reproduces a known failure pattern in current sub-1B VLMs in which color names are unreliable (e.g., naming "brown" for a gray object). This matters for accessibility applications (e.g., blind-user assistance) where color naming is safety-relevant. Any such downstream use would need to route color-specific queries through a dedicated color-calibrated model.

Equity. Physical robotics kits cost money; a browser-native demo that runs on any laptop lowers the hardware barrier, but the specialist still requires teacher-curated training data. A robust teacher-side workflow is needed to prevent a kit that works for some classrooms' object taxonomies and fails for others. The specialist's transfer-learning design (train a new head in minutes on ~250 images per class) is explicitly an equity choice.

Model citation. LFM2.5-VL-450M is released by Liquid AI under the LFM 1.0 License, which permits commercial use under $10M annual revenue. This project is educational and clearly within that scope.

Appendix A: Reproducibility

```
git clone https://github.com/jonasneves/aipi540-tabletop-perception
cd aipi540-tabletop-perception
pip install -r requirements.txt
make dataset   # downloads + stages Caltech-101, ~2 min
make eval      # runs all three models + exports ONNX
make sync      # copies artifacts into public/
make serve     # local demo on :8088
```

Appendix B: AI Tooling

Claude Code (Opus 4.7) was used extensively during development for dataset-pipeline debugging (fixing torchvision's dead Caltech URL), code generation (naive/classical baseline scaffolds), and report drafting. Specific Claude-generated content is the scaffold of each model script and the initial structure of this report; all content has been reviewed, modified, and tested by the author. Model numbers, experiment design, and failure analysis come from real runs reproduced in `results/*.json` and `results/vlm_ood/*.json`.